

Phase 7: Project Documentation

7.1 Project Overview

LegalEase is an AI-powered legal document generator. Users provide a document type, the parties involved, terms and an effective date. The system drafts a formal, sectioned legal document using Google Gemini, shows an editable preview, and exports the result as TXT, DOCX or PDF with branding.

7.2 Installation Guide

Prerequisites

- Python 3.10 or above and pip
- A Gemini API key (from Google AI Studio)
- Git (to clone the repository)

Steps

```
# 1. Clone the repository
git clone <your-public-repo-url>
cd LegalEase

# 2. Create and activate a virtual environment
python -m venv venv
venv\Scripts\activate                # Windows
# source venv/bin/activate           # Linux / macOS
```

```
# 3. Install dependencies
pip install -r requirements.txt

# 4. Configure environment variables
echo GEMINI_API_KEY=your_key_here > .env

# 5. Start the backend (terminal 1)
uvicorn legalEaseAPI.main:app --reload

# 6. Start the frontend (terminal 2)
streamlit run frontend/app.py
```

Open <http://localhost:8501> in the browser.

7.3 Configuration

Variable	Where	Purpose	Default
GEMINI_API_KEY	.env	Authenticates with the Gemini API	none (required)
GEMINI_MODEL	.env / config.py	Model used for generation	gemini-1.5-pro (update if retired)
BACKEND_URL	.env / config.py	FastAPI base URL used by Streamlit	http://localhost:8000

7.4 User Guide

Step	Action
------	--------

1	Enter the Document Type , e.g. Freelance Work Contract, NDA, Lease Agreement, Employment Offer Letter
2	Enter Parties Involved with roles, e.g. Jane Doe (Service Provider), TechNova Inc. (Client)
3	Enter Terms & Conditions separated by semicolons, e.g. Work must be delivered by May 15, 2025; Payment within 7 days of invoice; Confidentiality must be maintained at all times
4	Enter the Effective Date , e.g. April 15, 2025
5	Click Generate Document and wait for the success message
6	Review the styled preview
7	(Optional) Click Click to Edit Document , change the text, press Ctrl+Enter to apply
8	Download as .TXT , .DOCX or .PDF
9	Complete any [placeholders] and have the document reviewed by a legal professional before use

7.5 API Reference

Method	Endpoint	Description
GET	/	Health check
POST	/generate	Generate a legal document
GET	/docs	Interactive Swagger UI (auto-generated by FastAPI)

POST /generate request fields: document_type (str), parties (str), terms (str), dates (str). Response: {"document": "<text>"}.

7.6 Code Documentation Summary

File	Purpose
legalEaseAPI/main.py	Creates the FastAPI app, registers routes, defines /
legalEaseAPI/routes.py	DocumentRequest model and POST /generate
ai_core/ gemini_generator.py	Builds the prompt and calls Gemini
ai_core/generator.py	Sanitizing, HTML preview, DOCX and PDF creation
frontend/app.py	Streamlit interface
config.py	Paths, model name, backend URL
requirements.txt	Pinned dependencies
run.sh	Convenience script to start both servers

7.7 Troubleshooting / FAQ

Problem	Likely cause	Solution
"Could not generate the document" in UI	Backend not running	Start Uvicorn; check BACKEND_URL
500 error from /generate	Invalid API key, quota exceeded, or model unavailable	Check .env, wait and retry, update GEMINI_MODEL
ModuleNotFoundError: ai_core	App started from the wrong folder	Run commands from the project root

PDF shows ? for some characters	Latin-1 limitation of FPDF	Avoid special symbols or upgrade to fpdf2 with a Unicode font
Logo not shown	Wrong path or missing image	Check Image/Logo.png and Image/inverseLogo.png
Port already in use	Another process on 8000 / 8501	Stop it or run on another port (--port 8001)

7.8 Limitations

- Output is AI-generated and may contain errors, omissions or jurisdiction-specific gaps; **not legal advice**
- Requires internet access and a valid API key; subject to API rate limits
- Some details (addresses, jurisdiction, amounts) appear as placeholders unless included in the inputs
- No user accounts, history storage or e-signatures
- Classic FPDF has Latin-1 character limitations

7.9 Future Enhancements

- Additional input fields (addresses, jurisdiction, compensation amounts) and dropdown of document types
- Contract analysis: plain-language summaries and highlighted key clauses for uploaded documents
- Multilingual document generation and translation
- User accounts, saved drafts and document history
- Unicode PDF support (fpdf2 + TTF fonts), customizable branding (upload own logo/fonts)
- Cloud deployment (Render / Railway / Fly.io backend, Streamlit Community Cloud frontend) with Docker/Procfile

- Integration with legal databases and clause libraries; e-signature workflow

7.10 Deployment Notes (Future)

- Add `requirements.txt` with pinned versions, and a `Dockerfile` or `Procfile` (web: `uvicorn legalEaseAPI.main:app --host 0.0.0.0 --port $PORT`)
- Store `GEMINI_API_KEY` as a platform environment variable, not in the repository
- Set `BACKEND_URL` in the frontend to the deployed backend URL

7.11 Conclusion

LegalEase shows how Generative AI can lower the barrier to producing structured legal documents. By combining a FastAPI backend, Gemini-powered drafting and a Streamlit interface with editing and multi-format export, it lets founders, freelancers, landlords and individuals create professional first drafts in minutes.

7.12 References

- FastAPI documentation: <https://fastapi.tiangolo.com>
- Streamlit documentation: <https://docs.streamlit.io>
- Google Generative AI (Gemini) documentation: <https://ai.google.dev>
- python-docx documentation: <https://python-docx.readthedocs.io>
- FPDF documentation: <https://pyfpdf.readthedocs.io>
- Python documentation: <https://docs.python.org/3/>